

Are C++ Contracts Ready to Ship in C++26?

Document number: P4043R0

Date: 2026-03-07

Authors: Darius Neațu <dariusn@adobe.com>

Audience: EWG, WG21

Abstract

The C++ Contracts facility introduced by P2900 represents a major addition to the C++ language, intended to support specification of program correctness through preconditions, postconditions, and assertions.

Since its adoption into the C++ Working Draft, the feature has continued to generate substantial design discussion within the committee and the broader C++ community. Several papers have raised concerns about aspects of the design, and multiple proposals intended to address these concerns have not yet reached consensus.

This paper explores whether C++ Contracts are ready to ship in C++26 and suggests reconsidering the shipping vehicle for the feature. Deferring the feature to a future vehicle — such as C++29 or a White Paper — would allow additional time to reach stronger technical consensus and ensure a stable design for such a significant language facility.

1. Introduction

Before finalizing shipment in C++26, the committee may wish to explicitly evaluate whether the current design of C++ Contracts has reached sufficient maturity and consensus for standardization.

This paper therefore proposes a simple question for EWG: whether the C++ Contracts facility, in its current form, is ready to ship in C++26.

Contracts are intended to provide a standardized way for C++ programmers to express program requirements and guarantees through preconditions, postconditions, and assertions. The goal is to improve program correctness.

The facility was adopted into the Working Draft through P2900 during the Hagenberg 2025 meeting. Since that time, the design has remained the subject of significant discussion within the committee.

Several recent papers have raised concerns about the semantics, safety implications, practical deployability of the feature, and teachability. At the same time, proposals intended to address specific aspects of the design have not yet achieved consensus.

Following the decision in mid-2025 to conclude the work of SG21, EWG has been responsible for the continued discussion and evaluation of Contracts.

It is sometimes argued that reconsidering the shipping vehicle of a feature late in the standardization cycle may be difficult. However, an important goal of the C++ standardization process is to ship stable and widely supported designs; in many cases this means standardizing practices that already exist in real-world codebases.

When substantial design discussion remain, reconsidering the timing of a feature can improve the long-term quality and adoption of the resulting standard.

2. Current situation

The current state of Contracts facility in the C++26 Working Draft can be summarized as follows:

1. C++ Contracts were adopted into the Working Draft through P2900 during the Hagenberg 2025 meeting.
2. The feature represents a substantial addition to the C++ language.
3. Ongoing design discussions continue within EWG.
4. Several proposals intended to address concerns about the design have been presented, but none has yet reached consensus.

Taken together, these observations suggest that the design continues to evolve and remains an active area of discussion within the committee.

2.1 National Body feedback

Contracts have also been the subject of National Body feedback - more than 20 NB comments related to C++26 Contracts (<https://github.com/cplusplus/nballot/issues?q=is%3Aissue%20state%3Aclosed%20state%3Aopen%20contract&page=1>).

In particular:

- comments **NL 17.10** (<https://github.com/cplusplus/nballot/issues/982>), **US 69-125** (<https://github.com/cplusplus/nballot/issues/704>), and **GB 04-124** (<https://github.com/cplusplus/nballot/issues/703>), raised concerns regarding the `evaluation_exception()` API. EWG accepted the issue and applied P3819R0 (<http://wg21.link/P3819R0>) in the Kona 2025 meeting.
- comment **AT 1-057** (<https://github.com/cplusplus/nballot/issues/632>) raised concerns regarding the detection of replaceability of contract violation handler. P3886R0 (<https://wg21.link/P3886R0>) fix was also applied in the Kona 2025 meeting.
- comment **RO 2-056** (<https://github.com/cplusplus/nballot/issues/631>) raised concerns regarding the reliability of contract enforcement semantics. This issue got consensus for pursuing a solution in the C++26 timeframe. Several proposals have attempted to address these concerns by introducing stronger enforcement guarantees - P3911R2 (<http://wg21.link/P3911R2>), P4009R0 (<http://wg21.link/P4009R0>), P4005R0 (<http://wg21.link/P4005R0>). At the time of writing, these proposals have not yet achieved consensus.

Some of these issues have been resolved, while others remain under active discussion. The presence of continuing NB discussion suggests that further design exploration may still be beneficial.

2.2 Precedent in WG21 process

WG21 has, on several occasions, reconsidered the timing of major language features when questions about design maturity or consensus arose.

Major language facilities sometimes require multiple iterations across standard cycles before reaching their final form. One well-known precedent is Concepts, which were originally targeted for C++11 but were removed from the draft and later reintroduced in a refined form for C++20 after additional design work.

This paper raises the question of whether the current Contracts design is ready to ship in C++26, or whether allowing additional time for further design work could improve the resulting feature.

3. Evidence of ongoing design discussions

3.1 Contracts related papers

Several recent papers have raised questions about aspects of the Contracts facility or explored alternative design directions.

References are listed in Section 8. Examples include:

- P3573R0 — *Contract concerns*
- P3829R0 — *Contracts do not belong in the language*
- P3835R0 — *Contracts make C++ less safe - full stop!*
- P3851R0 — *Position on contract assertions for C++26*
- P3853R0 — *A thesis+antithesis=synthesis rumination on Contracts*
- P3909R0 — *Contracts should go into a White Paper — even at this late point*
- P4015R0 — *Enforcing Contract Conditions with Statements*
- P4020R0 — *Concerns about contract assertions*

These papers present a wide range of viewpoints on topics such as safety, semantics, deployability, and long-term design direction.

Taken together, this body of work indicates that the design space for Contracts continues to be actively discussed within the committee.

3.2 Teachability and risk of misuse

Another consideration when shipping a major language facility is how easily the feature can be understood and used correctly by the broader C++ community.

Contracts introduce new semantics around evaluation modes, enforcement behavior, and interactions with program control flow. While these mechanisms are powerful, they may also be difficult to explain and reason about, particularly for developers encountering the facility for the first time.

Several papers and (reflector) discussions have noted that misunderstandings about the semantics of contract assertions could lead to incorrect assumptions about program behavior.

Because Contracts operate at the boundary between specification and runtime behavior, ensuring that the facility is teachable and difficult to misuse is particularly important.

Additional time for design iteration and real-world experimentation could help ensure that the feature is both technically robust and easier for developers to understand and apply correctly in practice.

Taken together, these observations suggest that the C++ Contracts design remains an active area of exploration, with multiple competing perspectives on safety, usability, and deployment models.

4. Considerations for shipping in C++26

The C++ Contracts facility represents a significant addition to the C++ language and may affect several aspects of the C++ ecosystem, including:

- library design
- tooling
- optimization
- safety and reliability
- large-scale software development

Because of this broad impact, the stability and clarity of the design are particularly important.

If substantial questions remain open, deferring the feature could allow the committee to incorporate additional design insights and implementation experience before finalizing the facility in the standard.

5. Possible paths forward

If the committee concludes that the C++ Contracts facility would benefit from additional design time, several alternative paths could be considered:

- deferring C++ Contracts to C++29
- publishing the design as a White Paper

Such an approach would allow additional time for:

- resolving outstanding concerns
- exploring alternative design directions
- gaining implementation and deployment experience
- building stronger consensus within the committee and the community

Importantly, this paper does not argue against C++ Contracts as a concept. Rather, it raises the question of whether additional time could help ensure a robust and widely accepted design.

The goal is to ensure that the eventual feature ships with strong technical consensus, practical implementation experience, and a stable design.

5.1 Impact of deferring the feature

Deferring C++ Contracts from C++26 would likely have little impact on existing code or implementations.

Implementations could continue to experiment with the facility outside the C++26 standard, allowing further design exploration and practical experience.

Deferral would therefore allow additional experimentation while reducing the risk of standardizing a facility whose design is still evolving.

6. Suggested polls

EWG is asked for guidance on the following question:

- Poll 0: Do we believe that the C++ Contracts facility is ready to ship in C++26?
- Poll 1: Should WG21 reconsider the shipping vehicle for C++ Contracts currently targeted for C++26?
- Poll 2: If so, should WG21 explore deferring Contracts to a future vehicle (e.g., C++29 or a White Paper) to allow additional design work and consensus building?

7. Conclusion

C++ Contracts represent an ambitious and potentially valuable feature for C++.

However, ongoing design discussion, concerns raised in recent papers, and feedback from National Bodies suggest that the committee may wish to consider whether C++26 is the most appropriate shipping vehicle.

Deferring the feature could allow WG21 additional time to refine the design, gather implementation experience, and build broader consensus before standardizing such a significant language facility.

8. References

8.1 C++26 Contracts facility

[1] P2900R14 — Joshua Berne, Timur Doumler, Andrzej Krzemiński, et al., "Contracts for C++", WG21. <https://wg21.link/P2900R14> (<https://wg21.link/P2900R14>) (published: 2025-02-13)

8.2 Design exploration proposals

[2] P3911R2 — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, Herb Sutter, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", WG21. <https://wg21.link/P3911R2> (<https://wg21.link/P3911R2>) (published: 2026-01-14)

[3] P3919R0 — Ville Voutilainen, "Guaranteed-(quick-)enforced contracts", WG21.

<https://wg21.link/P3919R0> (<https://wg21.link/P3919R0>) (published: 2025-11-05)

[4] P3946R0 — Andrzej Krzemieński, "Designing enforced assertions", WG21.

<https://wg21.link/P3946R0> (<https://wg21.link/P3946R0>) (published: 2025-12-14)

[5] P4015R0 — Joshua Berne, "Controlling Contract-Assertion Properties", WG21.

<https://wg21.link/P3400R2> (<https://wg21.link/P3400R2>) (published: 2025-12-14)

[6] P3912R0 — Timur Doumler, Joshua Berne, Gašper Ažman, Oliver Rosten, Lisa Lippincott, Peter Bindels, "Design considerations for always-enforced contract assertions", WG21.

<https://wg21.link/P3912R0> (<https://wg21.link/P3912R0>) (published: 2025-12-15)

[7] P4005R0 — Ville Voutilainen, "A proposal for guaranteed-(quick-)enforced contracts",

WG21. <https://wg21.link/P4005R0> (<https://wg21.link/P4005R0>) (published: 2026-02-02)

[8] P4009R0 — Ville Voutilainen, "A proposal for solving all of the contracts concerns", WG21.

<https://wg21.link/P4009R0> (<https://wg21.link/P4009R0>) (published: 2026-02-09)

[9] P4015R0 — Lisa Lippincott, "Enforcing Contract Conditions with Statements", WG21.

<https://wg21.link/P4015R0> (<https://wg21.link/P4015R0>) (published: 2026-02-10)

8.3 Papers arguing against shipping Contracts in C++26

[10] P4020R0 — Andrzej Krzemieński, "Concerns about contract assertions", WG21.

<https://wg21.link/P4020R0> (<https://wg21.link/P4020R0>) (published: 2026-02-23)

[11] P3909R0 — Ville Voutilainen, "Contracts should go into a White Paper - even at this late point", WG21. <https://wg21.link/P3909R0> (<https://wg21.link/P3909R0>) (published: 2025-11-02)

[12] P3853R0 — Ville Voutilainen, "A thesis+antithesis=synthesis rumination on Contracts",

WG21. <https://wg21.link/P3853R0> (<https://wg21.link/P3853R0>) (published: 2025-10-05)

[13] P3851R0 — J. Daniel Garcia, Jose Gomez, Raul Huertas, Javier Lopez-Gomez, Jesus Martinez, Francisco Palomo, Victor Sanchez, "Position on contract assertions for C++26",

WG21. <https://wg21.link/P3851R0> (<https://wg21.link/P3851R0>) (published: 2025-09-29)

[14] P3835R0 — John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, "Contracts make C++ less safe – full stop!", WG21. <https://wg21.link/P3835R0> (<https://wg21.link/P3835R0>) (published: 2025-09-03)

[15] P3829R0 — David Chisnall, John Spicer, Ville Voutilainen, Gabriel Dos Reis, Jose Daniel Garcia Sanchez, "Contracts do not belong in the language", WG21. <https://wg21.link/P3829R0> (<https://wg21.link/P3829R0>) (published: 2025-07-15)

[16] P3573R0 — Michael Hava, J. Daniel Garcia Sanchez, Ran Regev, Gabriel Dos Reis, John Spicer, Bjarne Stroustrup, J.C. van Winkel, David Vandevoorde, Ville Voutilainen, "Contract concerns", WG21. <https://wg21.link/P3573R0> (<https://wg21.link/P3573R0>) (published: 2025-01-12)